

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

Facultad de Ingeniería de Sistemas e Informática



CURSO: Lenguajes y Compiladores

DOCENTE: Prudencio Vidal, Javier Antonio

GRUPO: 5

INTEGRANTES:

Arboleda Sanchez, Ericka Sofia	24200047
Badillo Castillo, Edwin Piero	24200150
Ccoicca Fernandez, Adrian	24200154
Huamanchahua Huayanay, Oscar	24200161
Maque Espinoza, Steveen Dennys	24200059
Reynoso Castillo, José Luis	24200169
Zarate Vilchez, Axhell Jesus	24200177

Índice

Índice.....	2
1. Introducción.....	3
2. Gramática Formal del Lenguaje Dummy (Semana 6).....	4
2.1. Definición de la gramática en BNF.....	4
2.3. Manejo de ambigüedades.....	7
2.4. Manejo de errores sintácticos.....	8
3. Implementación del Parser en Bison (Semana 7).....	8
3.1. Estructura del archivo .y.....	8
3.2. Declaración de tokens.....	9
3.3. Reglas de producción.....	10
4. Integración Lexer + Parser (Semana 8).....	10
4.1. Conexión entre Flex y Bison.....	10
4.2. Instrucciones de compilación.....	11
4.3. Pruebas de ejecución.....	12
5. Conclusiones.....	12

1. Introducción

El presente informe documenta el desarrollo de las fases correspondientes a las semanas 6, 7 y 8 del proyecto del compilador del lenguaje Dummy. El análisis sintáctico es la segunda etapa del proceso de compilación. Su objetivo es verificar que la secuencia de tokens producida por el analizador léxico siga las reglas gramaticales del lenguaje. Mientras que el lexer responde a la pregunta de si cada símbolo del código fuente es válido, el parser responde si la combinación de esos símbolos tiene sentido estructuralmente.

Para llevar a cabo esta fase, se utiliza la herramienta Bison, que es el equivalente para la gramática de lo que Flex es para el análisis léxico. Bison lee un archivo con extensión `y` que contiene las reglas gramaticales escritas en notación BNF y genera automáticamente el código C del analizador sintáctico.

Las tres semanas cubiertas en este informe corresponden a las siguientes actividades según el cronograma del proyecto:

- **Semana 6:** Definición formal de la gramática del lenguaje Dummy en papel, incluyendo la identificación de todos los no terminales, terminales y reglas de producción.
- **Semana 7:** Implementación de la gramática en el archivo `parser.y` de Bison, incluyendo la declaración de tokens, la precedencia de operadores y las acciones semánticas básicas.
- **Semana 8:** Integración real del lexer generado por Flex con el parser generado por Bison, eliminación de los `printf` del lexer y validación del parsing completo con programas de prueba.

2. Gramática Formal del Lenguaje Dummy (Semana 6)

La gramática formal de un lenguaje de programación define con precisión las estructuras sintácticas válidas que pueden aparecer en un programa. Se expresa mediante reglas de producción en notación BNF (Backus-Naur Form), donde cada regla describe como un símbolo no terminal puede expandirse en una secuencia de terminales y no terminales.

2.1. Definición de la gramática en BNF

A continuación se presenta la gramática completa del lenguaje Dummy. Los símbolos en mayúsculas representan terminales (tokens reconocidos por el lexer) y los símbolos en minúsculas representan no terminales (estructuras que se deben expandir).

```
programa      -> KW_INICIO DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER
```

```
sentencias    -> sentencia sentencias  
               | /* vacio */
```

```
programa -> KW_INICIO DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER  
          | /* vacio */
```

```
sentencia     -> declaracion  
               | asignacion  
               | condicional  
               | ciclo  
               | impresion  
               | retornar  
               | comentario  
               | error DELIM_PUNTO_COMA
```

```
declaracion   -> tipo IDENTIFICADOR OP_ASIGNACION expresion DELIM_PUNTO_COMA  
               | tipo IDENTIFICADOR DELIM_PUNTO_COMA
```

```
tipo          -> KW_NUM | KW_DEC | KW_TEXT
```

```
asignacion    -> IDENTIFICADOR OP_ASIGNACION expresion DELIM_PUNTO_COMA
```

```

condicional -> KW_CUANDO DELIM_PAR_IZQ condicion DELIM_PAR_DER
DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER
          | KW_CUANDO DELIM_PAR_IZQ condicion DELIM_PAR_DER
DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER KW_SINO DELIM_LLAVE_IZQ
sentencias DELIM_LLAVE_DER

ciclo      -> KW_LOOP DELIM_PAR_IZQ condicion DELIM_PAR_DER
DELIM_LLAVE_IZQ sentencias DELIM_LLAVE_DER

impresion -> KW_IMPRIMIR DELIM_PAR_IZQ expresion DELIM_PAR_DER

retorno -> KW_RETORNAR expresion DELIM_PUNTO_COMA
          | KW_RETORNAR DELIM_PUNTO_COMA

expresion  -> expresion OP_OR expresion
          | expresion OP_AND expresion
          | OP_NOT expresion
          | expresion OP_IGUAL expresion
          | expresion OP_DIFERENTE expresion
          | expresion OP_MAYOR expresion
          | expresion OP_MENOR expresion
          | expresion OP_MAYOR_IGUAL expresion
          | expresion OP_MENOR_IGUAL expresion
          | expresion OP_SUMA expresion
          | expresion OP_RESTA expresion
          | expresion OP_MULT expresion
          | expresion OP_DIV expresion
          | expresion OP_MOD expresion
          | OP_RESTA expresion
          | DELIM_PAR_IZQ expresion DELIM_PAR_DER
          | factor

factor     -> IDENTIFICADOR | LIT_ENTERO | LIT_DECIMAL
          | LIT_TEXT | LIT_BOOLEANO

```

2.2. Descripción de cada regla gramatical

- **programa:** Es el símbolo inicial de la gramática. Todo programa válido en Dummy debe comenzar con la palabra reservada inicio seguida de un bloque de sentencias encerrado entre llaves. La estructura es: inicio { sentencias }.

- **sentencias:** Define la secuencia de instrucciones dentro de un bloque. Se define de forma recursiva por la derecha (sentencia sentencias), lo que permite que un bloque contenga cualquier cantidad de sentencias consecutivas. La producción vacía permite bloques sin instrucciones.
- **condicion:** Representa las expresiones booleanas que controlan los condicionales y bucles. Puede ser una comparación relacional (expresion operador_rel expresion), una negación lógica (! condicion), una combinación lógica (condicion && condicion o condicion || condicion), o un literal booleano directamente.
- **sentencia:** Engloba todos los tipos de instrucciones válidas del lenguaje. Cada alternativa corresponde a una construcción diferente del lenguaje Dummy.
- **declaracion:** Permite declarar una variable especificando su tipo, nombre e, opcionalmente, un valor inicial. Por ejemplo: num x = 10; o dec promedio;
- **asignacion:** Permite cambiar el valor de una variable ya declarada. Por ejemplo: x = x + 1;
- **condicional:** Implementa la estructura de decisión del lenguaje. Puede tener solo la rama verdadera (cuando) o incluir también la rama falsa (sino).
- **ciclo:** Implementa la repetición mediante la estructura loop. El ciclo se ejecuta mientras la condición entre paréntesis sea verdadera. El cuerpo del ciclo es un bloque de sentencias encerrado entre llaves.
- **impresion:** Permite mostrar un valor o mensaje por consola usando la función imprimir. El punto y coma lo agrega la regla sentencia al incluir impresion como alternativa.

- **retorno:** Permite retornar un valor desde el bloque principal. Puede retornar una expresión o no retornar ningún valor. Al igual que impresion, el punto y coma lo agrega la regla sentencia.
- **expresion:** Es la regla más importante de la gramática. Define todas las combinaciones válidas de operadores y operandos. Las expresiones pueden ser aritméticas, relacionales y lógicas, y pueden anidarse con paréntesis.
- **factor:** Representa los valores atómicos que no se descomponen más: identificadores de variables, literales numéricos, literales de texto y valores booleanos.

2.3. Manejo de ambigüedades

Un problema clásico en las gramáticas es la ambigüedad en las expresiones con varios operadores. Por ejemplo, la expresión $2 + 3 * 4$ puede interpretarse como $(2 + 3) * 4 = 20$ o como $2 + (3 * 4) = 14$. Para resolver esto, Bison permite declarar la precedencia y asociatividad de los operadores.

```
%left  OP_OR                -- menor precedencia
%left  OP_AND
%left  OP_IGUAL OP_DIFERENTE OP_MAYOR OP_MENOR OP_MAYOR_IGUAL OP_MENOR_IGUAL
%left  OP_SUMA OP_RESTA
%left  OP_MULT OP_DIV  OP_MOD
%right OP_NOT                -- mayor precedencia
```

Los operadores declarados al final tienen mayor precedencia. %left indica asociatividad izquierda ($a - b - c$ se evalúa como $(a - b) - c$) y %right indica asociatividad derecha. La declaración %right OP_NOT al final le otorga la mayor precedencia, lo que significa que el operador lógico ! se evalúa antes que cualquier

operador aritmético o relacional. El operador menos unario no está implementado en la versión actual del parser.

```
La lista oficial de tokens con tipos semánticos es: %token <str>
IDENTIFICADOR, %token <str> LIT_TEXT, %token <str> LIT_DECIMAL, %token
<entero> LIT_ENTERO, %token <entero> LIT_BOOLEANO
```

2.4. Manejo de errores sintácticos

Bison ofrece el token especial error que permite al parser recuperarse de un error sintáctico sin detener el análisis por completo. Se incluye la siguiente regla dentro de sentencia:

```
yyerror reporta el error con numero de linea y aborta el analisis
```

Cuando el parser detecta un error sintáctico, la función yyerror imprime el número de línea exacto donde ocurrió el problema y detiene el análisis de inmediato. Esto evita propagar errores y da al programador retroalimentación precisa. El programa concluye con el mensaje ANALISIS CONCLUIDO CON ERRORES SINTACTICOS.

3. Implementación del Parser en Bison (Semana 7)

La semana 7 corresponde a la traducción de la gramática formal definida en papel a un archivo funcional de Bison (.y). Este archivo es procesado por Bison para generar automáticamente el código C del analizador sintáctico..

3.1. Estructura del archivo .y

Al igual que los archivos de Flex, los archivos de Bison tienen tres secciones separadas por el delimitador `%%`:

```
Seccion 1:Codigo C auxiliar + declaracion de tokens
%%
Seccion 2:Reglas gramaticales con acciones
%%
Seccion 3:Funciones adicionales (main, yyerror)
```

La primera sección contiene el código C que se copia al inicio del archivo generado, incluyendo los includes necesarios y la declaracion de todos los tokens mediante directivas `%token`. La segunda sección contiene las reglas de producción en formato BNF extendido. La tercera sección contiene las funciones de soporte.

3.2. Declaración de tokens

En la primera sección del archivo `.y` se declaran todos los tokens que el parser espera recibir del lexer. Estos deben coincidir exactamente con los tokens que el archivo `lexer.l` retorna:

```
%token KW_INICIO KW_CUANDO KW_SINO KW_LOOP
%token KW_NUM KW_DEC KW_TEXT
%token KW_IMPRIMIR KW_RETORNAR
%token IDENTIFICADOR
%token LIT_ENTERO LIT_DECIMAL LIT_TEXT LIT_BOOLEANO
%token OP_ASIGNACION OP_SUMA OP_RESTA OP_MULT OP_DIV OP_MOD
%token OP_IGUAL OP_DIFERENTE OP_MAYOR OP_MENOR
%token OP_MAYOR_IGUAL OP_MENOR_IGUAL
%token OP_AND OP_OR OP_NOT
%token DELIM_PAR_IZQ DELIM_PAR_DER
%token DELIM_LLAVE_IZQ DELIM_LLAVE_DER DELIM_PUNTO_COMA
%token TOKEN_COMENT TOKEN_ERROR
```

Al ejecutar Bison con la opción -d, se genera automáticamente el archivo parser.tab.h que contiene las definiciones numéricas de cada token. Este archivo es incluido por el lexer mediante `#include "parser.tab.h"`, lo que garantiza que ambos compartan los mismos códigos de token.

3.3. Reglas de producción

Las reglas de producción en Bison siguen este formato:

```
no_terminal
    : alternativa1 { accion C; }
    | alternativa2 { accion C; }
    ;
```

Las acciones entre llaves son código C opcional que se ejecuta cuando el parser reconoce esa regla. En la implementación actual, las acciones imprimen mensajes de confirmación para facilitar la depuración y la demostración al docente.

4. Integración Lexer + Parser (Semana 8)

La semana 8 corresponde a la conexión real entre el analizador léxico generado por Flex y el analizador sintáctico generado por Bison. Esta integración es la que convierte los dos módulos independientes en un compilador funcional de la fase de análisis.

4.1. Conexión entre Flex y Bison

El mecanismo de conexión funciona de la siguiente manera. Cuando Bison necesita el siguiente token, llama internamente a la función `yylex()`. Esta función es justamente la que Flex genera a partir del archivo `lexer.l`. Por lo tanto, al compilar

ambos archivos juntos, el parser obtiene los tokens directamente del lexer sin necesidad de un intermediario.

Para que el lexer conozca los códigos numéricos de cada token, debe incluir el archivo de cabecera generado por Bison:

```
#include "parser.tab.h"
```

Este include es la única línea que conecta los dos archivos. A partir de ese momento, cuando el lexer hace return KW_INICIO, el parser recibe el número entero que corresponde a ese token y lo procesa según sus reglas gramaticales.

4.2. Instrucciones de compilación

La compilación del proyecto integrado requiere ejecutar cuatro pasos en orden. Todos los comandos se ejecutan desde la terminal de Windows (cmd o PowerShell) dentro de la carpeta del proyecto:

```
# Paso 1: Bison genera parser.tab.c y parser.tab.h
bison -d parser.y

# Paso 2: Flex genera lex.yy.c
flex lexer.l

# Paso 3: GCC compila ambos archivos juntos
gcc parser.tab.c lex.yy.c -o compilador.exe

# Paso 4: Ejecutar con un programa de prueba
.\compilador.exe ejemplo1.g5z80
```

El proyecto incluye el script compilar.bat que automatiza los cuatro pasos anteriores. Para ejecutarlo basta con escribir .\compilar.bat en la terminal de

Windows. El script verifica que cada paso se complete sin errores antes de continuar al siguiente.

4.3. Pruebas de ejecución

Se incluyen los archivos `ejemplo1.g5z80` y `ejemplo_con_error.g5z80`. El primero contiene un programa completo en el lenguaje Dummy que ejercita todas las construcciones: declaraciones de variables, condicionales con y sin rama sino, bucle loop, uso de imprimir y retornar, y comentarios con la sintaxis `%% ... %%`. La salida esperada para un programa válido es la confirmación de cada construcción reconocida seguida del mensaje `ANALISIS SINTACTICO COMPLETADO EXITOSAMENTE`.

El segundo archivo demuestra el manejo de errores: contiene un error sintáctico intencional (`num y = ;`) que provoca que el parser reporte la línea exacta del error y concluya con `ANALISIS CONCLUIDO CON ERRORES SINTACTICOS`.

Para demostrar el manejo de errores, se puede modificar el archivo de prueba introduciendo intencionalmente errores sintácticos como omitir un punto y coma o una llave de cierre. En ese caso el parser reportará el error con el número de línea correspondiente y, gracias a la regla de recuperación, continuará analizando el resto del programa.

5. Conclusiones

El desarrollo de las semanas 6, 7 y 8 permitió comprender la diferencia fundamental entre el análisis léxico y el análisis sintáctico. Mientras el primero verifica que cada símbolo sea válido individualmente, el segundo verifica que la combinación de símbolos tenga sentido gramatical.

La herramienta Bison facilita enormemente la implementación de un parser porque permite expresar la gramática de forma declarativa mediante reglas BNF, sin necesidad de codificar manualmente los algoritmos de parsing. Bison genera internamente un parser LALR(1) que es eficiente y robusto.

La integración entre Flex y Bison se logra mediante un mecanismo simple: el archivo de cabecera `parser.tab.h` que comparte los códigos de token entre ambas herramientas. Este patrón de diseño es el estándar de la industria y es el mismo que se utiliza en compiladores reales.

El proyecto se encuentra en condiciones de avanzar hacia la siguiente fase, que corresponde al análisis semántico y la construcción del árbol de sintaxis abstracta (AST), tal como indica el cronograma para las semanas 9 y 10.